

Flask & MongoDB Assignment

DevOps Assignment — Tutedude

Submitted By: Umair Khan

Date: September 27, 2026

Course: DevOps Training — Tutedude

GitHub Repository:

<https://github.com/Umair-bey/tutedude-assignments/tree/main/Assignment-3>

Table of Contents

1. Objective
 2. Environment Setup
 3. Dependencies Installed
 4. MongoDB Atlas Setup
 5. Task 1 — JSON API Route
 6. Task 2 — Frontend Form with MongoDB Atlas
 7. Flask Request Logs
 8. Security Practices
 9. How to Run
 10. Conclusion
-

1. Objective

Build a Flask application with:

- **A JSON API route** (`/api`) that reads data from a backend file and returns it as a JSON list.
- **A frontend HTML form** that submits data to MongoDB Atlas:
 - **On success:** redirect to another page displaying **"Data submitted successfully"**.
 - **On error:** display the error on the same page without redirection.

Technologies Used

Component	Version
Python	3.14.2
Flask	3.1.3
PyMongo	4.18.1
dnspython	2.8.0
python-dotenv	1.2.3
MongoDB Atlas	Free M0 tier
Editor	VS Code
Shell	PowerShell / Command Prompt

2. Environment Setup

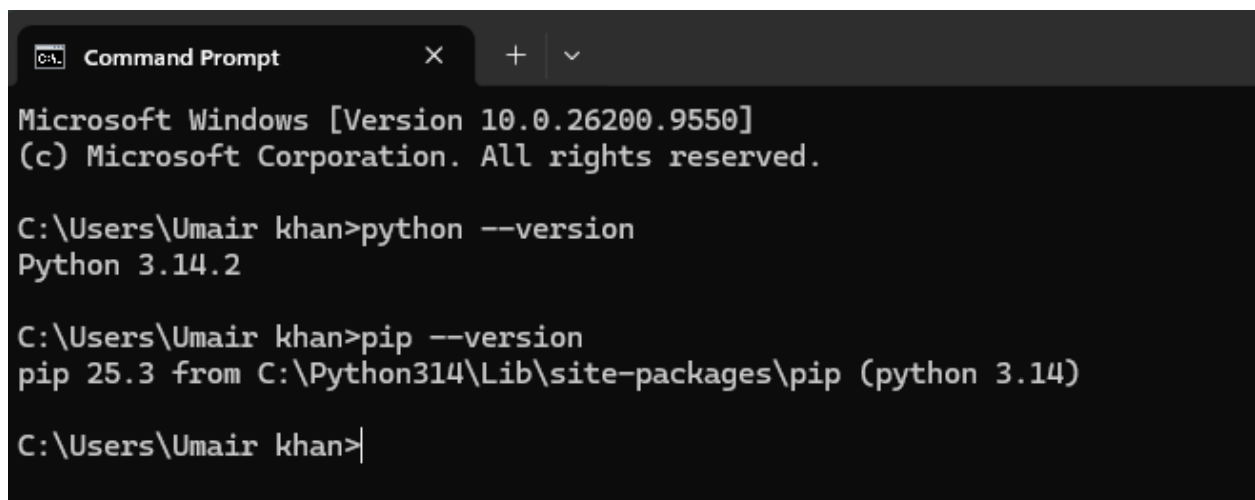
2.1 Python & pip Verification

Python 3.14.2 and pip 25.3 were already installed on the system. Verified with:

text

python --version

pip --version



```
Microsoft Windows [Version 10.0.26200.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Umar khan>python --version
Python 3.14.2

C:\Users\Umar khan>pip --version
pip 25.3 from C:\Python314\Lib\site-packages\pip (python 3.14)

C:\Users\Umar khan>
```

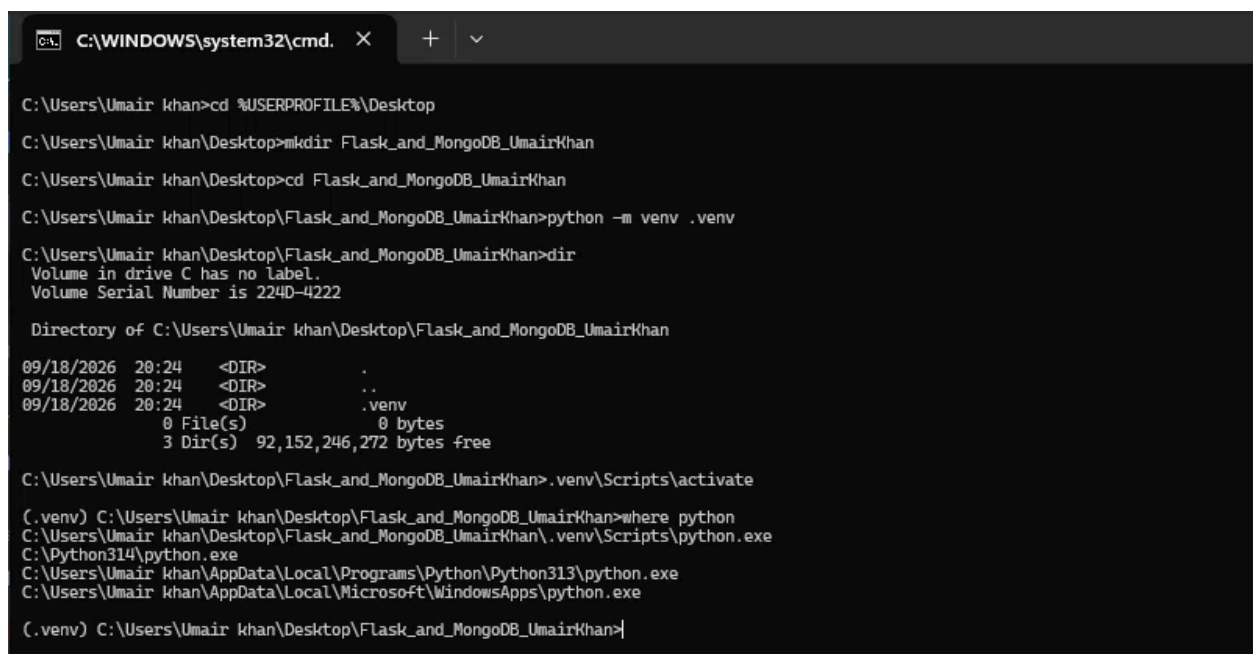
Figure 1: Python 3.14.2 and pip 25.3 verified

2.2 Project Folder & Virtual Environment

Created project folder `Flask_and_MongoDB_UmairKhan` on the Desktop, then created an isolated virtual environment to prevent dependency conflicts.

text

```
cd %USERPROFILE%\Desktop
mkdir Flask_and_MongoDB_UmairKhan
cd Flask_and_MongoDB_UmairKhan
python -m venv .venv
.venv\Scripts\activate
where python
```



```
C:\WINDOWS\system32\cmd. X + v

C:\Users\Umair khan>cd %USERPROFILE%\Desktop
C:\Users\Umair khan\Desktop>mkdir Flask_and_MongoDB_UmairKhan
C:\Users\Umair khan\Desktop>cd Flask_and_MongoDB_UmairKhan
C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan>python -m venv .venv
C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan>dir
Volume in drive C has no label.
Volume Serial Number is 224D-4222

Directory of C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan

09/18/2026  20:24    <DIR>        .
09/18/2026  20:24    <DIR>        ..
09/18/2026  20:24    <DIR>        .venv
               0 File(s)              0 bytes
               3 Dir(s)  92,152,246,272 bytes free

C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan>.venv\Scripts\activate
(.venv) C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan>where python
C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan\.venv\Scripts\python.exe
C:\Python314\python.exe
C:\Users\Umair khan\AppData\Local\Programs\Python\Python313\python.exe
C:\Users\Umair khan\AppData\Local\Microsoft\WindowsApps\python.exe

(.venv) C:\Users\Umair khan\Desktop\Flask_and_MongoDB_UmairKhan>
```

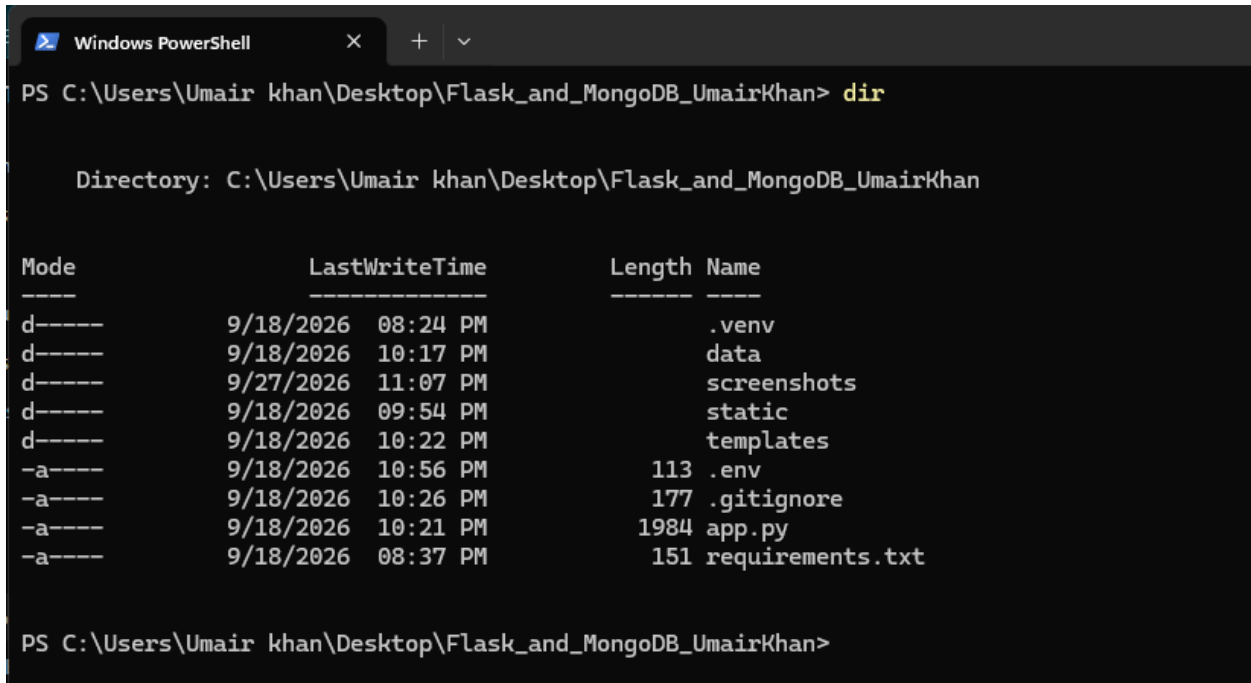
Figure 2: Virtual environment created and activated

2.3 Final Folder Structure

text

```
Flask_and_MongoDB_UmairKhan/
├── .env                (MongoDB URI — gitignored)
├── .gitignore          (excludes .env, .venv, __pycache__)
├── .venv/              (virtual environment)
├── app.py              (Flask application)
├── requirements.txt     (Python dependencies)
├── data/
│   └── data.json       (backend file for Task 1)
├── screenshots/        (evidence for this doc)
└── static/             (reserved for assets)
```

```
└─ templates/
   └─ index.html      (form page)
   └─ success.html    (success page)
```



```
Windows PowerShell
PS C:\Users\Umais khan\Desktop\Flask_and_MongoDB_UmaisKhan> dir

Directory: C:\Users\Umais khan\Desktop\Flask_and_MongoDB_UmaisKhan

Mode                LastWriteTime         Length Name
----                -
d-----          9/18/2026   08:24 PM                .venv
d-----          9/18/2026   10:17 PM                data
d-----          9/27/2026   11:07 PM            screenshots
d-----          9/18/2026   09:54 PM                static
d-----          9/18/2026   10:22 PM            templates
-a-----          9/18/2026   10:56 PM           113 .env
-a-----          9/18/2026   10:26 PM           177 .gitignore
-a-----          9/18/2026   10:21 PM          1984 app.py
-a-----          9/18/2026   08:37 PM           151 requirements.txt

PS C:\Users\Umais khan\Desktop\Flask_and_MongoDB_UmaisKhan>
```

Figure 3: Required folders created via mkdir

3. Dependencies Installed

Installed via pip (inside the activated venv):

```
text
pip install flask pymongo dnspython python-dotenv
pip freeze > requirements.txt
```

```
C:\WINDOWS\system32\cmd. X + v
(.venv) C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan>pip install flask pymongo dnspython
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pymongo
  Downloading pymongo-4.18.1-cp314-cp314-win_amd64.whl.metadata (10 kB)
Collecting dnspython
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading pymongo-4.18.1-cp314-cp314-win_amd64.whl (821 kB)
822.0/822.0 kB 6.3 MB/s 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, dnspython, click, blinker, werkzeug, pymongo, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 dnspython-2.8.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 pymongo-4.18.1 werkzeug-3.1.8
[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan>
```

Figure 4: Flask, PyMongo, and dnspython installed successfully

```
Windows PowerShell X + v
PS C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan> .\venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan> pip show python-dotenv
Name: python-dotenv
Version: 1.2.3
Summary: Read key-value pairs from a .env file and set them as environment variables
Home-page:
Author:
Author-email: Saurabh Kumar <me+github@saurabh-kumar.com>
License: BSD-3-Clause
Location: C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan\.venv\Lib\site-packages
Requires:
Required-by:
(.venv) PS C:\Users\Umais khan\Desktop\Fask_and_MongoDB_UmaisKhan>
```

Figure 5: python-dotenv installed successfully

4. MongoDB Atlas Setup

4.1 Account & Free Cluster

- Created a free MongoDB Atlas account (Google Sign-In).
- Deployed a free **M0 cluster** on **AWS**, region **Mumbai (ap-south-1)**.
- Cluster name: **Cluster0**.

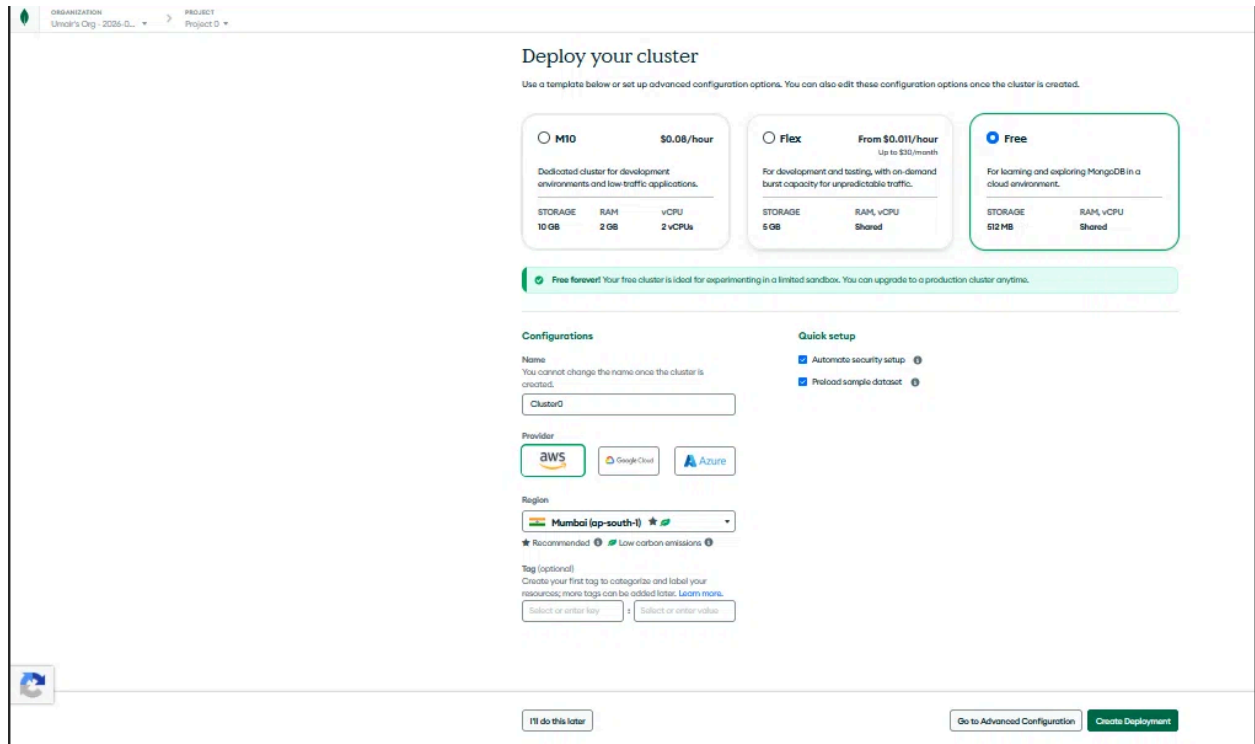


Figure 6: Free cluster configuration (Free tier, AWS, Mumbai)

4.2 Database User

Created a database user with read/write access: `umair2025khanin_db_user`.

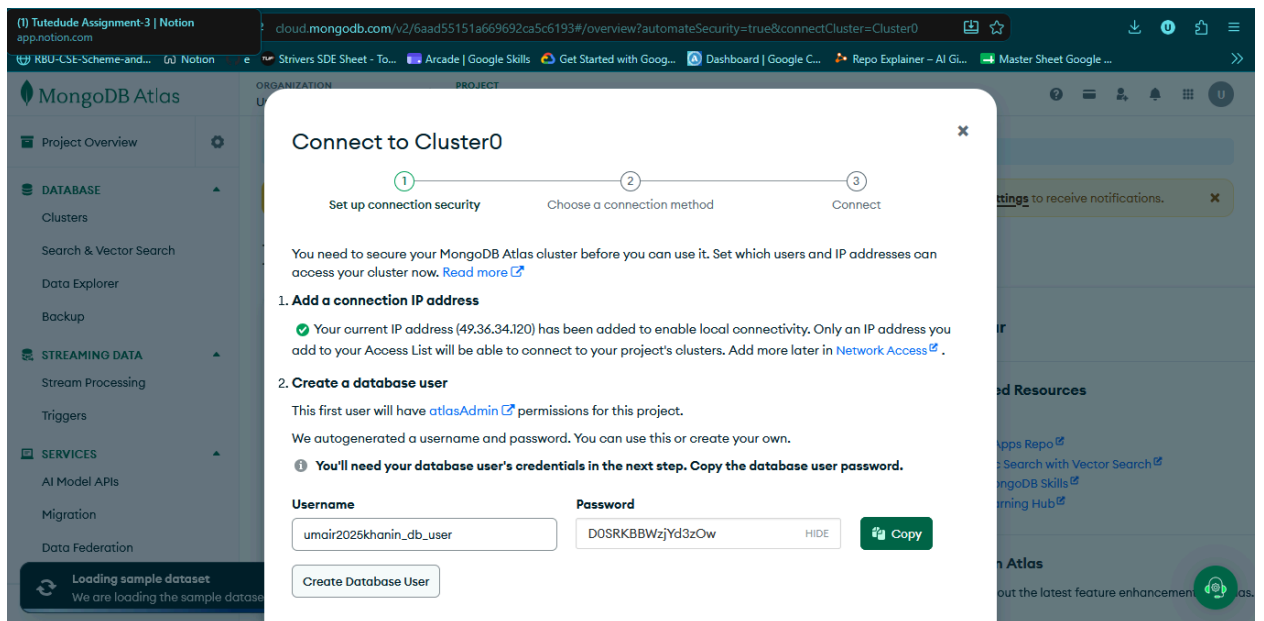


Figure 7: Database user credentials created

4.3 Network Access

Two IP rules were configured:

IP	Purpose	Status
49.36.34.120 /32	Current IP (auto-setup)	Active
0.0.0.0/0	Allow from anywhere (development)	Active

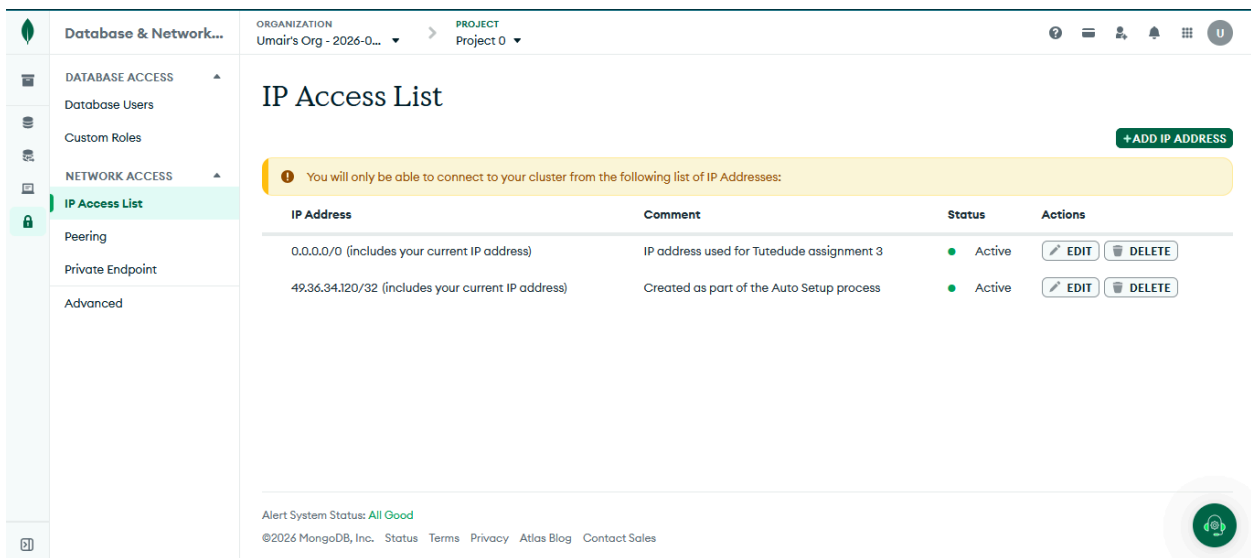


Figure 8: Network access whitelist

4.4 Connection String

Format:

text
mongodb+srv://umair2025khanin_db_user:<password>@cluster0.dvcs5w9.mongodb.net/?appName=Cluster0

Stored securely in a .env file (never committed to GitHub).

Connect to Cluster0

✓
Set up connection security

✓
Choose a connection method

3
Connect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver

Version

Python

4.7 or later

2. Install your driver

Run the following on the command line

Note: Use appropriate Python 3 executable

```
python -m pip install "pymongo[srv]"
```

[View MongoDB Python Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String

☒ Show Password

☐ View full code sample

```
mongodb+srv://umair2025khanin_db_user:DB_PASSWORD@cluster0.dvcs5w0.mongodb.net/?appName=Cluster0
```

The password for `umair2025khanin_db_user` is included in the connection string for your first time setup. This password will not be available again after exiting this connect flow.

RESOURCES

[Get started with the Python Driver](#)

[Python Starter Sample App](#)

[Access your Database Users](#)

[Troubleshoot Connections](#)

Go Back

Done

Figure 9: Connection string (password hidden)

4.5 Cluster Active

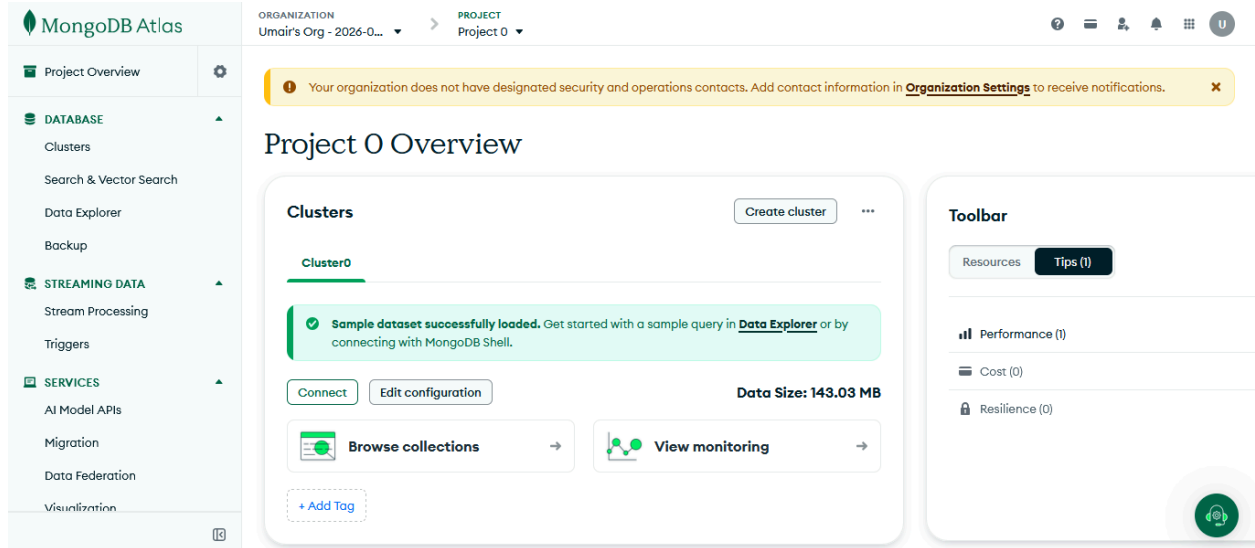


Figure 10: Atlas dashboard — Cluster0 active

5. Task 1 — JSON API Route

5.1 Requirement

Create a Flask application with an `/api` route that returns a JSON list. The data must be stored in a backend file, read from it, and returned as a response.

5.2 Backend File — `data/data.json`

```
json
[
  {"id": 1, "name": "Priyanshu", "role": "DevOps Engineer"},
  {"id": 2, "name": "Umair Khan", "role": "Tutodude Student"},
  {"id": 3, "name": "Alice", "role": "Developer"},
  {"id": 4, "name": "Bob", "role": "Designer"}
]
```

5.3 Implementation — `app.py` (Task 1 portion)

```
python
import os, json
from flask import Flask, jsonify

DATA_FILE = os.path.join(os.path.dirname(__file__), 'data', 'data.json')

def read_data_file():
```

```
with open(DATA_FILE, 'r') as f:
    return json.load(f)
```

```
@app.route('/api', methods=['GET'])
def api():
    data = read_data_file()
    return jsonify(data)
```

5.4 Explanation

- The `/api` route only accepts **GET** requests.
- It reads `data/data.json` using Python's built-in `json` module.
- It returns the parsed data using Flask's `jsonify()`, which sets **Content-Type: application/json**.
- The route is **stateless** — each request reads the file fresh from disk.

5.5 Testing

Opened `http://127.0.0.1:5000/api` in the browser — the JSON array was displayed.

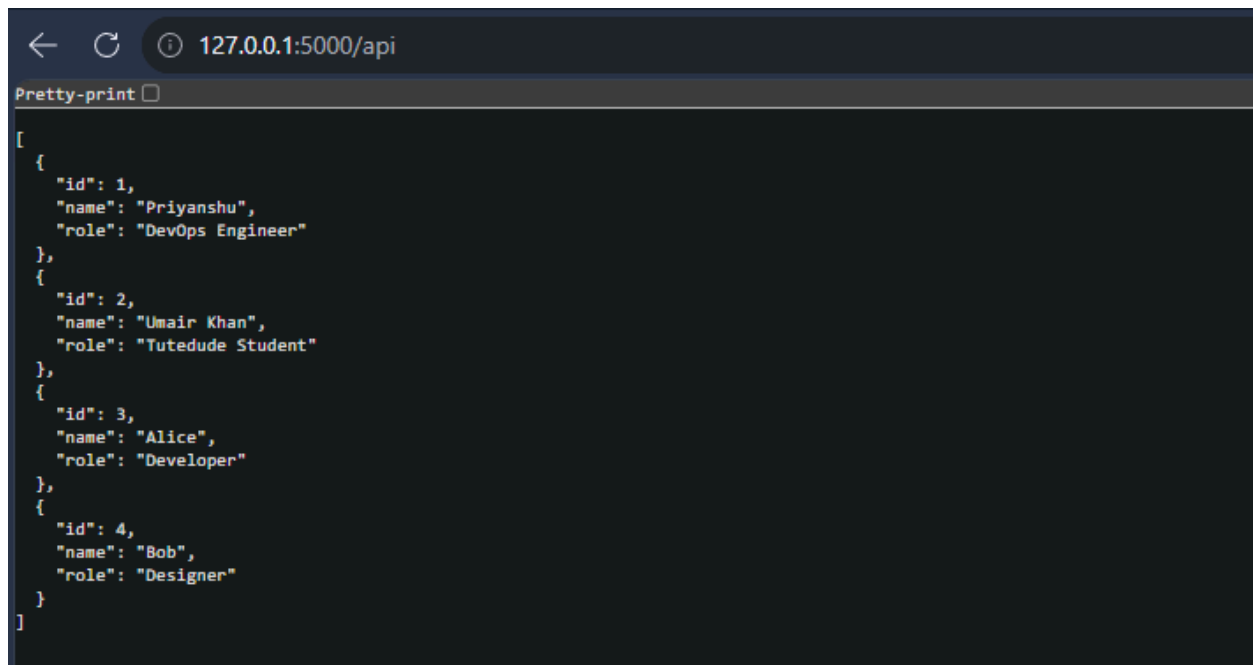


Figure 11: Task 1 — `/api` returns JSON list from backend file

6. Task 2 — Frontend Form with MongoDB Atlas

6.1 Requirements

- A frontend form that inserts submitted data into MongoDB Atlas.
- **On success:** redirect to another page displaying "Data submitted successfully".
- **On error:** display the error on the same page without redirecting.

6.2 Files Involved

- `templates/index.html` — form page
- `templates/success.html` — success page
- `app.py` — routes `/`, `/submit`, `/success`

6.3 Implementation — `app.py` (Task 2 portion)

```
python
from flask import render_template, request, redirect, url_for
from pymongo import MongoClient
from dotenv import load_dotenv
```

```
load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
client = MongoClient(MONGO_URI)
db = client.get_database('tutedude_db')
collection = db['submissions']
```

```
@app.route('/')
def index():
    return render_template('index.html')
```

```
@app.route('/submit', methods=['POST'])
def submit():
    try:
        data = request.form.to_dict()
        if not data.get('name') or not data.get('email'):
            raise ValueError("Name and Email are required.")
        collection.insert_one(data)
        return redirect(url_for('success'))
    except Exception as e:
        return render_template('index.html', error=str(e))
```

```
@app.route('/success')
def success():
    return render_template('success.html')
```

6.4 Explanation

- The `/` route renders the form.
- The form posts to `/submit` with `method="POST"`.
- On `/submit`:
 - Form data is extracted via `request.form.to_dict()`.
 - Input is validated (name and email required).
 - **On success:** record is inserted via `collection.insert_one()`, then Flask issues a **302 redirect** to `/success`.
 - **On error:** the same form page is re-rendered with the error message — **no redirect**.

6.5 Success Case — Test

Submitted the form at `http://127.0.0.1:5000/`. Browser redirected to `/success`.

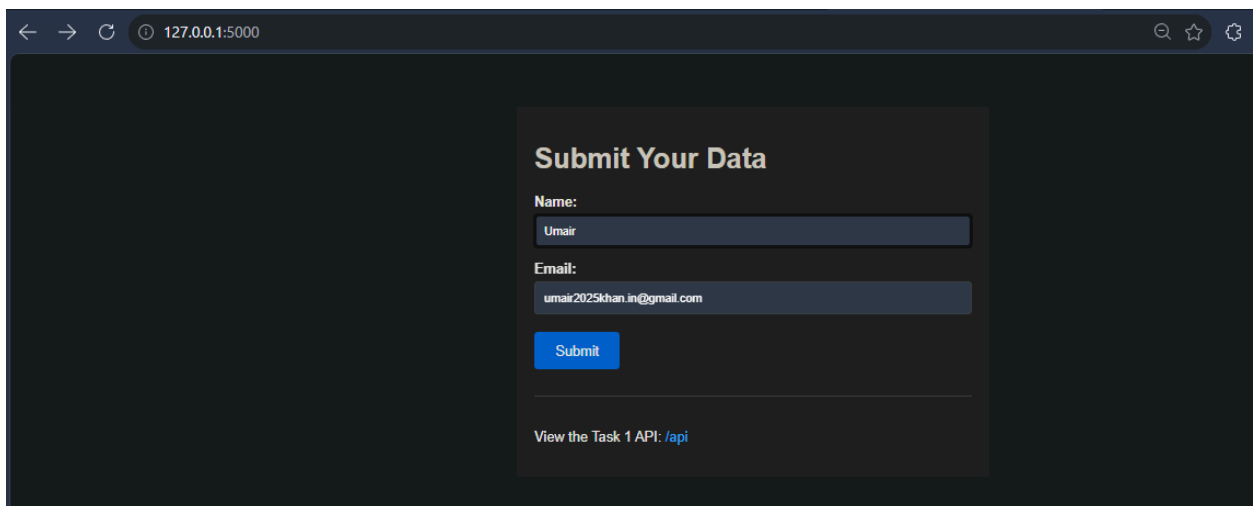
A screenshot of a web browser showing a form titled "Submit Your Data". The form has two input fields: "Name:" with the value "Umar" and "Email:" with the value "umar2025khan.in@gmail.com". Below the fields is a blue "Submit" button. At the bottom of the form, there is a link that says "View the Task 1 API: /api". The browser's address bar shows "127.0.0.1:5000".

Figure 12: Task 2 — form page

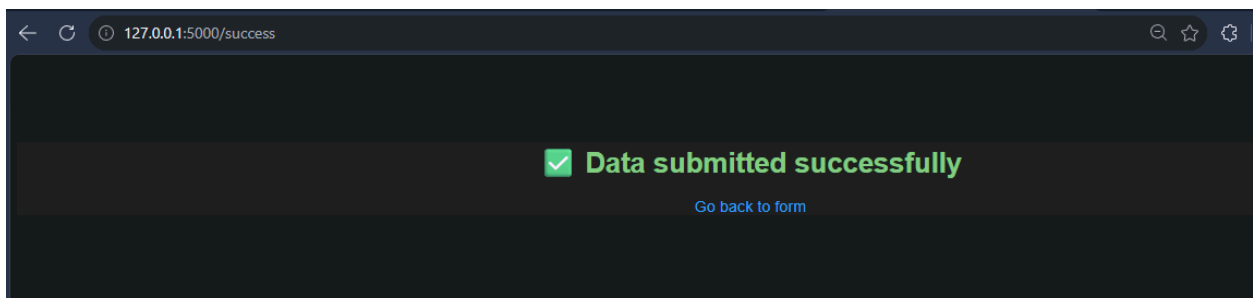


Figure 13: Task 2 — success page after valid submission

6.6 Error Case — Test

Temporarily replaced the password in `.env` with an invalid value, restarted Flask, and submitted the form. The browser showed a red error box on the **same page** (no redirect).

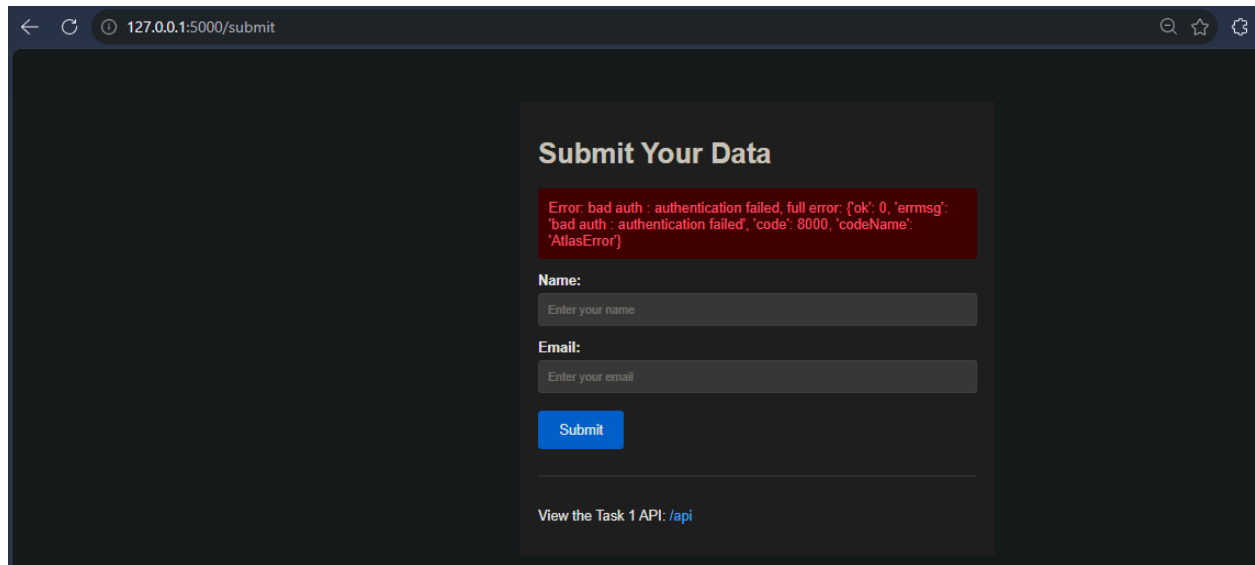


Figure 14: Task 2 — error displayed on same page (URL stays on `/submit`)

6.7 Data Verified in MongoDB Atlas

Opened MongoDB Atlas → Data Explorer → `tutede_db` → `submissions`. Documents from the form submission were visible.

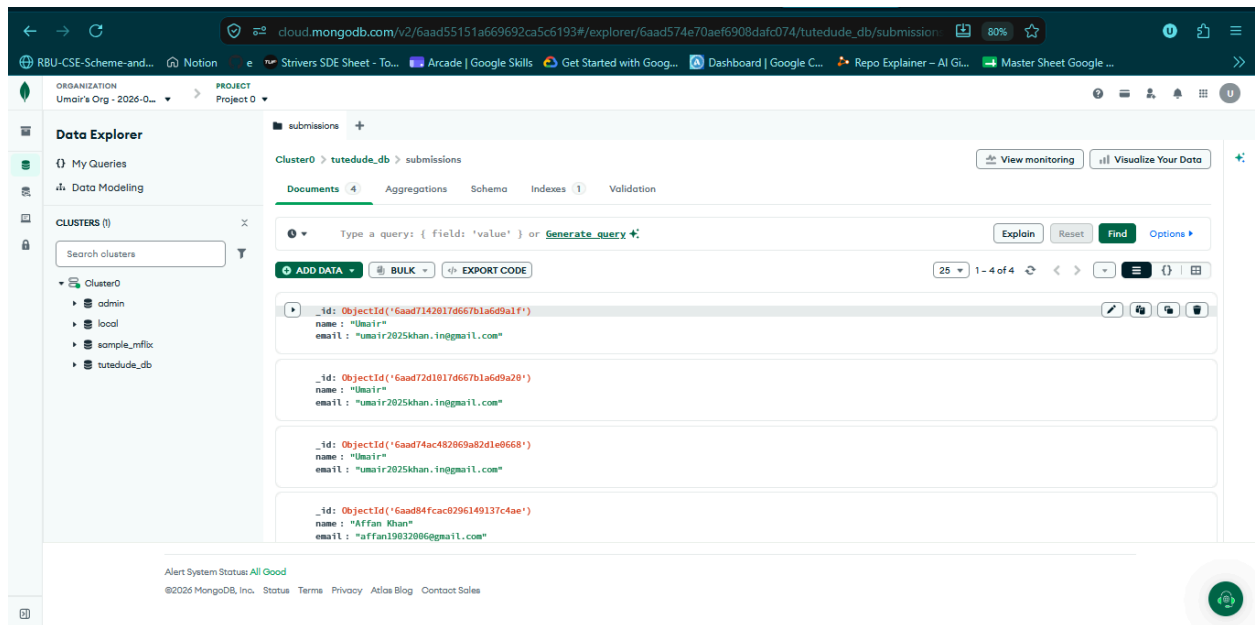


Figure 15: Documents inserted into MongoDB Atlas

7. Flask Request Logs

The Flask terminal captured the full request sequence during testing:

text

```
127.0.0.1 -- [...] "GET / HTTP/1.1" 200 -
127.0.0.1 -- [...] "POST /submit HTTP/1.1" 302 - ← Redirect fired
127.0.0.1 -- [...] "GET /success HTTP/1.1" 200 -
127.0.0.1 -- [...] "GET /api HTTP/1.1" 200 -
```

Line	Status	Meaning
GET / 200	OK	Form page loaded
POST /submit 302	Redirect	Task 2 success case — redirect issued
GET /success 200	OK	Success page loaded
GET /api 200	OK	Task 1 — API returned JSON

The **302 status code** on **POST /submit** is direct evidence that the redirect requirement for Task 2 was satisfied.

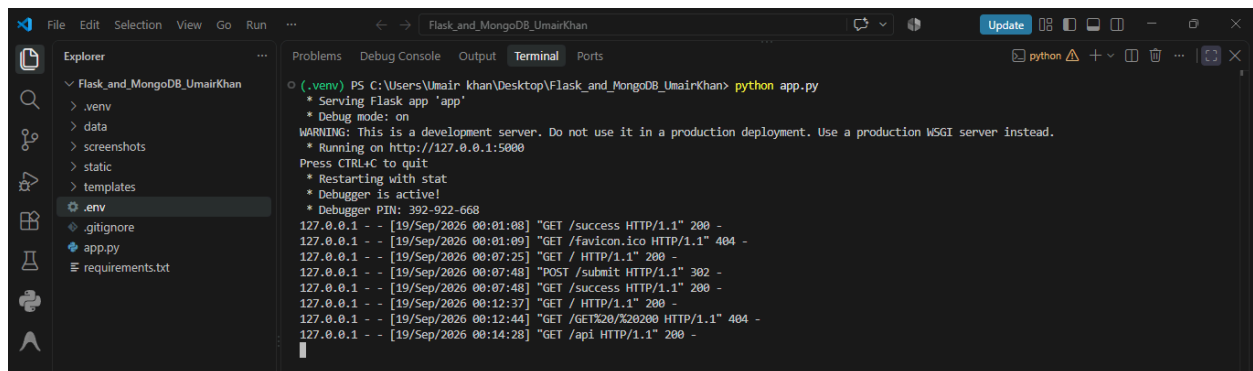


Figure 16: Flask request logs — POST /submit returns 302

8. Security Practices

- **Environment variables:** The MongoDB connection string is stored in a `.env` file (loaded via `python-dotenv`), not hardcoded in `app.py`.

- **.gitignore:** Excludes `.env`, `.venv/`, and `__pycache__/` from Git, so secrets and machine-specific files are never committed.
- **Password rotation:** The database user password was rotated via MongoDB Atlas → Database Access during development, minimizing risk from any exposure.

.gitignore contents:

```
text
# Environment variables (secrets)
.env

# Virtual environment
.venv/
venv/

# Python cache
__pycache__/
*.pyc

# VS Code
.vscode/

# OS files
.DS_Store
Thumbs.db
```

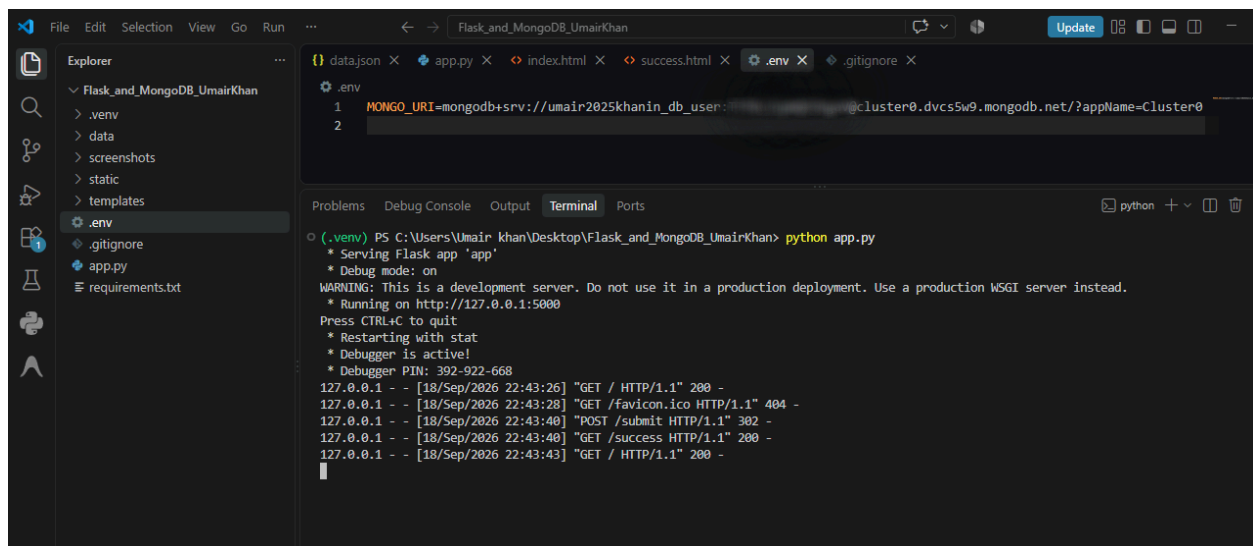


Figure 17: `.env` file containing `MONGO_URI` (sensitive — blurred in submission)

9. How to Run

Prerequisites: Python 3.10+, MongoDB Atlas account (free tier).

Steps:

Clone the repository:

text

```
git clone https://github.com/Umair-bey/Flask_and_MongoDB_UmairKhan.git
```

1. `cd Flask_and_MongoDB_UmairKhan`

Create and activate a virtual environment:

text

```
python -m venv .venv
```

2. `.venv\Scripts\activate`
 3. Install dependencies:
text
`pip install -r requirements.txt`
 4. Create a `.env` file with your own MongoDB URI:
text
`MONGO_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/?appName=Cluster0`
 5. Run the app:
text
`python app.py`
 6. Open in browser:
 - <http://127.0.0.1:5000/> — form page (Task 2)
 - <http://127.0.0.1:5000/api> — JSON API (Task 1)
-

10. Conclusion

Both tasks were successfully implemented, tested, and documented:

- **Task 1** — The `/api` route returns a JSON list read from `data/data.json`.
- **Task 2** — The frontend form inserts data into MongoDB Atlas, redirects to a success page on success (HTTP 302), and displays errors on the same page without redirecting (proved via forced authentication failure).

The project follows best-practice structure (virtual environment, environment variables, `.gitignore`), and all functionality is verified with terminal logs and browser screenshots.

GitHub Repository:

<https://github.com/Umaid-bey/tutedude-assignments/tree/main/Assignment-3>
